

AI Coding Agent Trust and Pull Request Outcomes in OSS

SENG 404 Interim Project Report — Group 6 / proj 6

Team: Kunwarbir Singh (V01024960), Gabriel Taves (V00939587), Amrinder Singh (V01024880), Luke Thomas (V00976989)

Date: June 7, 2026

How to Read This Interim Report

Sections marked **complete** contain usable results. Sections marked **in progress** contain early results, placeholder synthesis, or work that still needs robustness checks, trimming, figure polish, or citation verification. The final section gives the updated remaining-work timeline/Gantt chart.

Progress Summary

At this stage, the team has completed the shared dataset choice, most preprocessing, and the main empirical analyses for review sentiment, review engagement, and within-repository outcomes. The developer-experience and imbalance work has initial results but still needs final robustness checks and integration.

Completed or largely completed:

- Shared research framing and sub-question ownership.
- AIDev-pop dataset selection and table mapping.
- Agent-name normalization, timestamp parsing, PR outcome flags, and join-key checks.
- RQ1 sentiment analysis using VADER, TextBlob, a SentiCR-style classifier, and review-state proxies.
- RQ2 review-engagement framework and regression models.
- RQ3 within-repository acceptance, merge-speed, task-specialization, and size-specialization analysis.
- Initial RQ4 developer-experience and imbalance analysis using GitHub-enriched PR data.

Remaining before the final report:

- Verify and standardize all references.
 - Polish RQ4 robustness checks and write the section in final form.
 - Connect RQ2 and RQ3 more directly by testing whether low scrutiny predicts faster merge after controls.
 - Trim all individual sections into one unified paper.
 - Improve figures, captions, and table formatting.
 - Expand discussion, limitations, conclusion, and poster materials.
-

1. Introduction, Motivation, and Research Questions

1.1 Introduction

AI coding agents are increasingly participating in open-source software development by generating code, opening pull requests, and entering normal review workflows. Unlike private autocomplete tools, agent-authored pull requests become visible public artifacts. Maintainers must decide whether to review, request changes, approve, or merge these contributions.

This project studies how agent-authored pull requests are reviewed, discussed, and accepted in open-source repositories. The focus is real pull-request workflow data rather than benchmark performance, because open-source acceptance depends not only on whether code works, but also on reviewability, maintainer trust, repository norms, task type, author context, and reviewer discussion.

1.2 Motivation

Open-source maintainers do not evaluate AI coding agents through merge decisions alone. Review comments, requested changes, review duration, discussion patterns, and review effort also reveal trust, friction, concern, or approval. An agent may have a high merge rate but still receive negative technical feedback. Another may perform well only for certain task types or in certain repository contexts.

For that reason, this project treats AI-agent performance as a multi-dimensional problem. The goal is not to produce a simple leaderboard, but to understand how agent identity, task type, PR complexity, repository context, developer experience, review sentiment, and dataset imbalance interact in real open-source workflows.

1.3 Main Research Question

To what extent does the choice of AI coding agent affect pull request outcomes, including review sentiment, review engagement, acceptance, review duration, and merge success, across different task categories after accounting for pull request complexity, repository characteristics, developer experience, and dataset imbalance?

1.4 Team Research Questions and Status

ID	Area	Question	Owner	Interim Status
RQ1	Review sentiment	How does review sentiment toward AI coding agents differ across agents, and how has that sentiment evolved over time?	Amrinder	Complete; figure/caption polish remains
RQ2	Review engagement	How do reviewers engage with AI-authored PRs, and does engagement differ by agent, task type, and repository context?	Gabe	Complete; needs stronger link to RQ3
RQ3	Within-repository outcomes	In repositories where multiple agents are used, which agents achieve higher acceptance or faster merges, and do agents specialize by task type or PR size?	Luke	Complete; final integration remains
RQ4	Developer experience and robustness	Does developer experience influence how AI-generated PRs are reviewed and accepted after	Kunwarbir	In progress; initial controlled results available

		controlling for complexity, author association, and imbalance?		
Shared	Dataset imbalance	How do agent overrepresentation and uneven repository distribution affect conclusions?	Whole team	In progress; handled across sections

1.5 Coordination Note

RQ2 and RQ3 both study PR-level behavior, but their scopes are separated. RQ2 measures the review process: whether a PR is reviewed, how quickly review arrives, how much discussion occurs, and whether reviewers request changes. RQ3 measures outcomes: acceptance, merge speed, and specialization within repositories where multiple agents are used. The final report will connect them by testing whether lower scrutiny predicts faster merge after controlling for PR complexity and repository context.

2. Related Work and Background

Status: in progress. The structure is drafted and the working bibliography has more than the required 8-10 sources. The remaining work is to verify exact citation details, standardize formatting, and add short annotations where needed.

2.1 AI Coding Agents and Agent-Authored PRs

The AIDev dataset motivates this project because it makes agent-authored pull requests observable at scale. Instead of only evaluating tools through controlled coding benchmarks, AIDev allows analysis of how agents behave in public open-source workflows. Recent AIDev-based studies also show that agents differ by task type, change size, acceptance, and review outcome. This supports our decision to separate agent identity from task type, PR complexity, and repository context.

2.2 Pull Request Review in Open Source

Prior work on pull-request review shows that acceptance and latency depend on both technical and social factors. Contribution size, project norms, reviewer availability, author reputation, and repository context can all affect whether a PR is merged. This supports our choice to study not only merge success, but also review coverage, scrutiny, review duration, and discussion intensity.

Mining GitHub literature also warns that public repository data can be biased. Repositories differ widely, contributors are not randomly distributed, and repeated activity by the same author can distort global comparisons. This is directly relevant here because OpenAI Codex is heavily overrepresented and some agents are concentrated in extreme author-repository pairs.

2.3 Sentiment Analysis in Software Engineering

Sentiment analysis in software engineering is difficult because technical language can confuse general sentiment tools. Words such as "bug," "fail," "invalid," or "dead code" may describe technical issues rather than emotional negativity. Prior work on SentiCR, SentiStrength-SE, and SE-specific sentiment tools shows that method choice can change conclusions. Therefore, RQ1 compares general sentiment methods, a code-review-specific classifier, and a formal review-state proxy.

2.4 Developer Experience and Repository Context

Work on code-generation tools such as GitHub Copilot suggests that trust depends on workflow context, not only generated code quality. In open source, trust may also depend on who submits the PR, whether they are already associated with the repository, and whether the change is easy to review. This motivates RQ4, which asks whether developer experience, author association, PR complexity, and repository context explain outcomes beyond agent identity.

3. Methods and Dataset

3.1 Dataset

Status: complete.

The project uses **AIDev-pop**, the enriched subset of AIDev. AIDev-pop contains **33,596 agent-authored pull requests** from **2,807 repositories** with more than 100 stars. It includes pull-request metadata, discussion comments, review comments, review states, commits, task categories, repository data, issue data, and user/repository information.

The broader AIDev dataset contains **932,791 PRs** across **116,211 repositories**, but the team uses AIDev-pop because it includes the review activity and multi-agent coverage needed for the research questions.

Agent	PRs in AIDev-pop
OpenAI Codex	21,799
GitHub Copilot	4,970
Devin	4,827
Cursor	1,541
Claude Code	459

Dataset-wide outcomes include **24,014 merged**, **7,270 closed-unmerged**, and **2,312 open** PRs. Codex accounts for roughly 65% of AIDev-pop PRs, so imbalance is treated as a major validity concern.

3.2 Main Tables

Table	Main Use
<code>pull_request.parquet</code>	PR metadata, agent labels, repository ids, timestamps, merge status
<code>pr_comments.parquet</code>	General PR discussion comments
<code>pr_reviews.parquet</code>	Top-level review records and review states
<code>pr_review_comments_v2.parquet</code>	Inline code review comments
<code>pr_task_type.parquet</code>	LLM-classified task categories
<code>repository.parquet</code>	Repository-level context
<code>user.parquet, all_user.parquet</code>	User/account-level metadata

pr_commits.parquet, pr_commit_details.parquet	Commits, LOC, changed files, and complexity features
--	--

3.3 Shared Preprocessing

Status: complete.

Shared preprocessing includes normalizing raw agent names into five canonical agents, parsing timestamps, creating merged/rejected/open outcome flags, cleaning text for sentiment analysis, and checking join keys across PR, commit, task, review, and comment tables.

3.4 Analysis Strategy

The team compares agents across several outcome layers rather than relying on one raw metric. Shared strategies include agent-level descriptive statistics, task-category comparisons, PR complexity controls, repository-level and within-repository comparisons, developer-experience comparisons, and dataset imbalance checks.

RQ1 uses **21,922 human external review/comment text items** after removing bot comments, PR-author self-comments, empty comments, very short comments, code blocks, inline code, URLs, and generated markup. It compares VADER, TextBlob, a SentiCR-trained classifier, and review-state proxy signals.

RQ2 uses a two-stage Review Engagement Framework. Stage 1 measures whether each PR received human review. Stage 2 measures responsiveness, scrutiny, inline intensity, discussion intensity, and low-friction approval among reviewed PRs or with censoring where appropriate.

RQ3 restricts to repositories where at least two agents each have at least **k=5** or **k=10** PRs. This within-repository design helps control for repository-level confounds. It uses centered acceptance, paired Wilcoxon tests, fixed-effects logit models, Cox proportional hazards models, Jensen-Shannon divergence, and repo-fixed-effects size models.

RQ4 uses a GitHub-enriched subset of AIDev-pop. A total of **2,025 PR rows** were queried through the GitHub REST API, and **2,003 were successfully fetched**. GitHub enrichment provides additions, deletions, changed files, commits, author association, merge status, and PR state. Models use repository-clustered standard errors and exclude extreme high-volume author-repository pairs for robustness.

4. Preliminary Findings

4.1 RQ1: Review Sentiment Toward AI Coding Agents

Status: complete; final figure and caption polish remains.

Review sentiment toward AI coding agents is generally neutral-to-positive, but the ranking depends strongly on the sentiment method.

Sentiment Over Time

The strongest temporal pattern is that review discussion becomes less positive once the dataset reaches its high-volume period. The early December 2024 and August 2025 edge months are small and should be interpreted cautiously. In the main observed window, VADER and TextBlob are higher in January–April 2025 than in May–July 2025, while the SentiCR negative-comment share is slightly higher in May–July.

Period	Text Items	VADER Mean	TextBlob Mean	SentiCR Negative Share
Dec. 2024, sparse	41	0.665	0.160	9.8%
Jan–Apr. 2025	2,772	0.155	0.070	22.7%
May–Jul. 2025	18,978	0.095	0.056	23.8%
Aug. 2025, sparse	131	0.002	0.070	15.3%

Interpretation: Sentiment did not steadily improve over the observed period. Instead, general sentiment became more neutral during the high-volume months, and code-review-specific negativity remained around one quarter of comments. Because the sample is concentrated in May–July 2025, this should be treated as an initial temporal pattern rather than a long-term historical claim.

Sentiment by Agent — General Tone

General text sentiment ranks Claude Code highest and GitHub Copilot lowest:

Agent	VADER Mean	Positive Share	Negative Share
Claude Code	0.269	57.8%	13.4%
Cursor	0.147	48.5%	22.0%
Devin	0.141	39.0%	16.7%
OpenAI Codex	0.119	49.8%	23.3%
GitHub Copilot	0.080	36.6%	20.6%

Sentiment by Agent — Code-Review Specific and Formal Signals

Code-review-specific and formal review signals favor OpenAI Codex and Cursor:

Agent	SentiCR Neg. Share	Review-State Score	Approved	Changes Requested
OpenAI Codex	17.8%	0.613	66.5%	5.3%
Cursor	20.1%	0.592	63.3%	4.1%
Devin	22.3%	0.334	40.3%	6.9%
Claude Code	24.7%	0.366	41.1%	4.5%
GitHub Copilot	24.9%	0.239	36.0%	12.1%

Method Sensitivity

Agent rankings change depending on the sentiment method:

Method	Ranking (Most → Least Positive)
VADER	Claude Code > Cursor > Devin > OpenAI Codex > Copilot
TextBlob	Claude Code > Cursor > OpenAI Codex > Devin > Copilot
SentiCR-trained classifier	OpenAI Codex > Cursor > Devin > Claude Code > Copilot
Review-state proxy	OpenAI Codex > Cursor > Claude Code > Devin > Copilot

VADER differences are statistically significant (Kruskal-Wallis $H = 178.7$, $p \approx 1.4e-37$), and bootstrap checks keep Copilot last in 100% of samples.

Merged PRs show lower code-review negativity than unmerged PRs. The SentiCR negative share is 18.1% for merged PRs and 22.4% for unmerged PRs. In a logistic regression predicting merge outcome, SentiCR negativity predicts lower merge odds (coefficient = -0.479 , $p < 0.001$), while general VADER tone does not.

Interim interpretation: there is no single sentiment-based best-agent ranking. General tone, code-review-specific negativity, and formal review decisions capture different aspects of reviewer response.

4.2 RQ2: Review Engagement With AI-Authored Pull Requests

Status: Draft and much of analysis complete. Write-up needs to be flushed out with greater detail of the actual technical analysis -- need to figure out what depth of explanation is expected, but statistical methodology should be more comprehensively described and justified.

Coverage (4.2.1)

The review-engagement dataset contains **33,596 PRs** across **2,807 repositories**. Only **6,004 PRs (17.9%)** received at least one human review, making review coverage the first and most consequential finding.

Table 4.2.1

Agent	PRs	Human Review Coverage
OpenAI Codex	21,799	5.4%
Claude Code	459	24.6%
Cursor	1,541	25.5%
Devin	4,827	36.9%
GitHub Copilot	4,970	51.2%

96.4% of Codex PRs receive no human review. After controlling for task type, repository reputation, language, and commit count, Copilot, Devin, and Cursor are significantly more likely than Codex to receive human review.

After controlling for task type, repository reputation, language, and commit count, Copilot (OR = 5.99, 95% CI [3.81, 9.43]), Devin (OR = 3.63 [1.94, 6.77]), and Cursor (OR = 2.25 [1.19, 4.25]) are each significantly more likely than Codex to receive human review (all $p \leq .012$). Claude Code does not reach significance (OR = 1.79, $p = .054$), largely reflecting its small sample. chore and ci task types attract disproportionately low coverage regardless of agent; revert PRs are reviewed at higher rates.

Reviewer Engagement Profile (4.2.2)

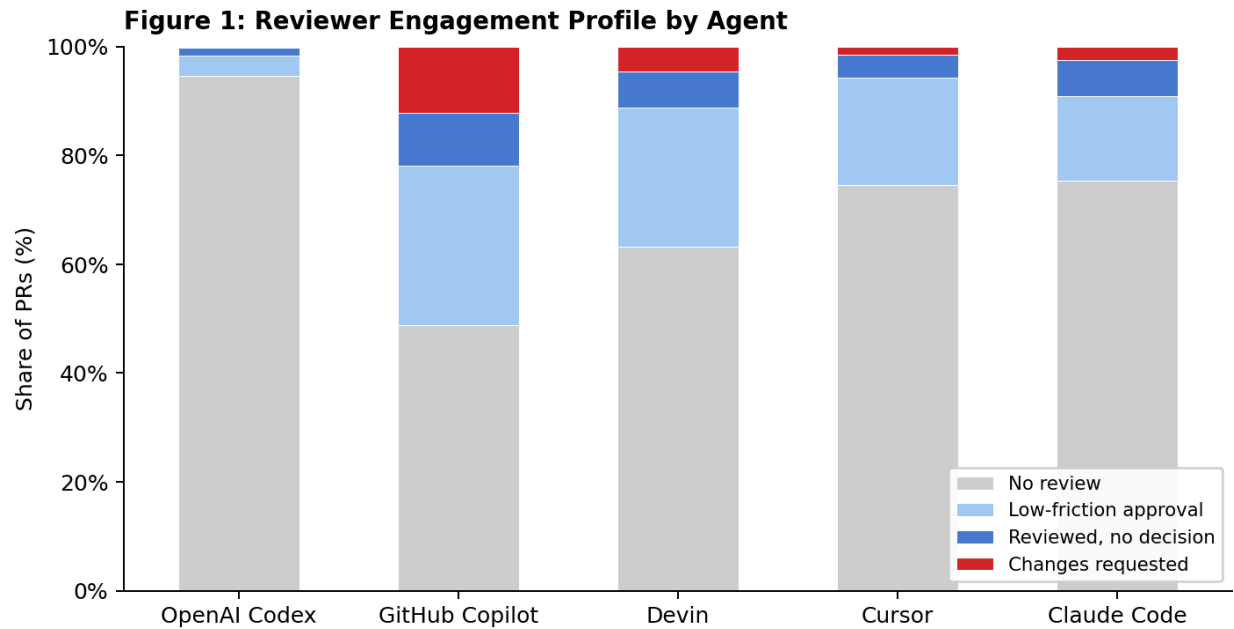


Figure 4.1 shows the full reviewer engagement spectrum per agent as a stacked bar. Codex's bar is dominated by the unreviewed segment. Among reviewed PRs, Copilot accumulates the most changes-requested outcomes, while Cursor has the highest share of low-friction approvals -- a pattern consistent with the within-repo acceptance results reported in RQ3. Figure 1 should be interpreted with the discrepancy in sample sizes shown in Table 4.2.1

Scrutiny (4.2.3)

Scrutiny ranges from **4.9% for Codex** to **23.8% for Copilot**. All scrutiny results are conditional on whether or not they received human review.

Table 4.2.2

Agent	Scrutiny Rate of Reviewed PRs
OpenAI Codex	4.9%
Cursor	5.6%
Claude Code	9.7%
Devin	12.5%
GitHub Copilot	23.8%

After controls, Copilot PRs are **3.40x** more likely than Codex PRs to receive **CHANGES_REQUESTED** (95% CI [2.30, 5.03], $p < .001$); Devin PRs are **1.81x** more likely ([1.11, 2.98], $p = .019$). Cursor and Claude Code do not differ significantly from Codex after repository context is controlled.

Reviewer Responsiveness (4.2.4)

Table 4.2.3

Agent	Median Time to First Review
Devin	1.0h
GitHub Copilot	1.4h
OpenAI Codex	2.3h
Cursor	3.6h
Claude Code	12.6h

Raw low-friction approval rates (approved with no CHANGES_REQUESTED) range from 57.1% for Copilot to 77.9% for Cursor. However, no agent coefficient in the low-friction approval model is statistically significant after controls (all $p > .05$). Agent identity alone does not predict friction-free approval once repository reputation, language, task type, and complexity are held constant. The raw differences reflect which repositories each agent operates in, not intrinsic agent properties.

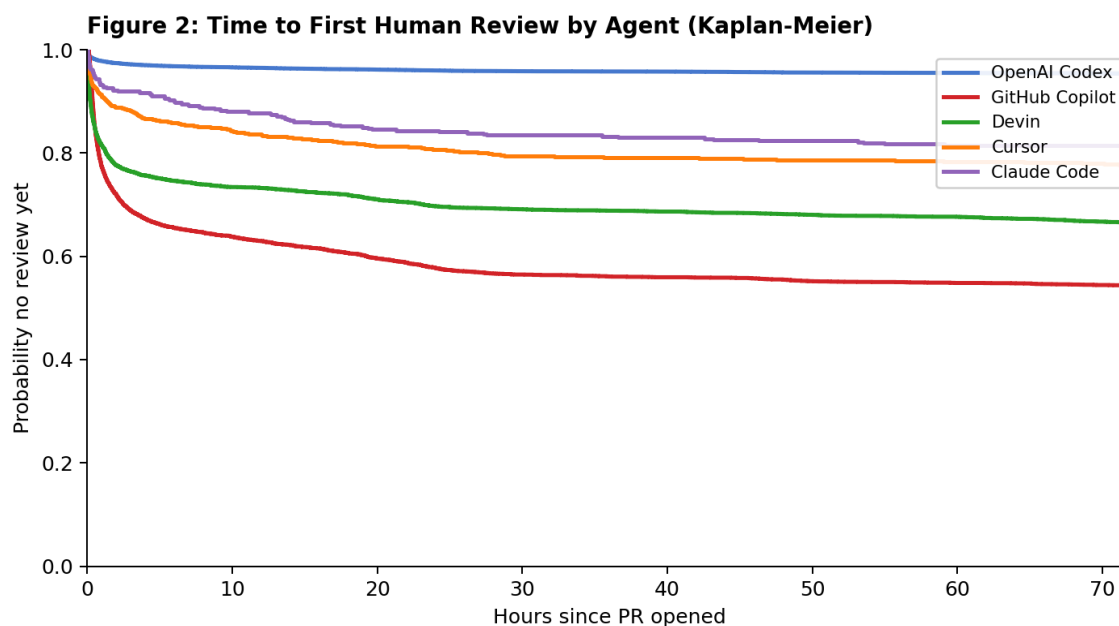


Figure 2 applies Kaplan-Meier survival estimation to time-to-first-human-review, treating the 82.1% of PRs that receive no review as right-censored observations. Each curve shows the probability that a PR from a given agent has not yet received human review at each elapsed hour; the plateau each curve settles into reflects that agent's unreviewed fraction rather than late-arriving review. Separation between agents is visible within the first three hours and persists across the full 72-hour window.

Interpretation of Findings (Interim) (4.2.5)

The five constructs together describe two distinct reviewer engagement modes: Codex operates largely outside the review system, while Copilot operates inside a stricter review culture -- highest coverage, fastest review, highest scrutiny, only non-zero discussion intensity, yet still majority low-friction approval. The weight of the evidence points to deployment context as a major driver of these differences: agents are not interchangeable across repositories, and repository norms significantly moderate how agents are treated. The null result on low-friction approval is the clearest indication that agent identity does not independently determine review quality once context is accounted for. These findings motivate the acceptance analysis in RQ3, where the same coverage and scrutiny patterns should predict downstream merge outcomes.

4.3 RQ3: Within-Repository Agent Comparison and Specialization

Status: complete; final integration remains.

The within-repository design is viable, especially at **k=5**.

Measure	k=5	k=10
Qualifying repositories	37	17
PRs in working panel	2,481	1,456
Codex present in repos	30	13
Cursor present in repos	21	10
Devin present in repos	17	11
GitHub Copilot present in repos	10	3
Claude Code present in repos	4	0

Acceptance

Centered acceptance compares each agent to its repository baseline.

Agent	Centered Acceptance k=5	Centered Acceptance k=10
Claude Code	+0.097	Not available
OpenAI Codex	+0.050	+0.014
Cursor	+0.011	+0.042
Devin	-0.047	-0.015
GitHub Copilot	-0.132	-0.148

The fixed-effects logit treats Codex as the reference agent. Both raw and adjusted odds ratios are shown below. The raw-to-adjusted gap is small and never flips sign, so the acceptance differences are not explained by task mix or PR size.

Agent	OR Raw k=5	OR Adjusted k=5	OR Raw k=10	OR Adjusted k=10
Cursor	1.92	2.01	2.94	3.23
Devin	0.72	0.78	0.88	0.94
GitHub Copilot	0.29	0.25	0.29	0.37

Merge Speed

Time-to-merge was modeled with a Cox model stratified by repository. A hazard ratio above 1 means faster merging than Codex.

Covariate	HR k=5	p k=5	HR k=10	p k=10
Cursor	1.45	<0.001	1.56	<0.001
Devin	0.78	0.001	0.77	0.003
GitHub Copilot	0.94	0.72	0.70	0.23
Claude Code	0.97	0.88	Not available	Not available
log LOC	0.87	<0.001	0.86	<0.001

Cursor merges fastest and Devin slowest across both thresholds. Larger PRs merge more slowly, with about a 13–14% lower merge hazard per unit of log LOC.

Task Specialization

Task-type specialization was measured using Jensen-Shannon divergence (JSD) between each agent's task mix and its repository's overall task mix. Higher JSD = more specialized.

Agent	JSD k=5	JSD k=10
Claude Code	0.381	Not available
Cursor	0.286	0.256
GitHub Copilot	0.264	0.322
OpenAI Codex	0.250	0.222
Devin	0.232	0.214

Task tendencies are interpretable but modest: Devin is feature-skewed, Copilot leans toward fixes and CI/build, Cursor does more docs and fixes, Codex is the most balanced, and Claude Code skews toward features and refactors on a small sample.

Size Specialization

Size specialization is clearer than task specialization. Size coefficients from the repo-fixed-effects regression on log LOC (reference: Codex):

Agent	Size Coef k=5	p k=5	Size Coef k=10	p k=10
Cursor	+0.72	<0.001	+0.67	<0.001
Devin	+0.88	<0.001	+0.74	<0.001
GitHub Copilot	-0.30	0.32	-0.17	0.72
Claude Code	+1.76	<0.001	Not available	Not available

Median raw LOC changed at k=5: Codex 57, Copilot 74, Devin 150, Cursor 162, Claude Code 620. This suggests that Codex tends toward smaller balanced contributions, while Cursor and Devin handle larger changes. Claude Code appears largest but has a thin sample.

Interim interpretation: inside the same repositories, agent choice is associated with different outcomes. Cursor has the strongest within-repository acceptance and merge-speed profile, GitHub Copilot has the weakest acceptance profile, and Devin merges slowest while concentrating more on feature work.

4.4 RQ4: Developer Experience, PR Complexity, and Repository Context

Status: in progress. Initial controlled models are available; robustness wording and final integration remain.

The developer-context analysis asks whether developer experience independently influences agent-assisted PR outcomes. Developer experience is measured using GitHub account age at PR creation, with followers as a secondary reputation proxy. The models also include agent, task category, PR complexity, author association, repository context, PR month, and repository-clustered standard errors.

Data Quality and High-Volume Account Check

A major issue is extreme author-repository concentration. Extreme pairs are defined as pairs with more than 100 observed PRs in AIDev, roughly the top 1% tail.

Agent	Extreme Pairs	PRs in Extreme Pairs	Total PRs	Extreme PR Share
Claude Code	2	3	458	0.7%
Cursor	3	112	1,541	7.3%
Devin	10	2,023	4,827	41.9%
OpenAI Codex	17	15,019	21,799	68.9%

Extreme author-repo concentration is a major confound. OpenAI Codex and Devin have especially large shares of PRs in extreme author-repo pairs, so naive global agent comparisons can overstate agent effects.

GitHub-Enriched PR Complexity Sample

In the GitHub-enriched sample (2,003 successfully fetched PRs), the median total churn is 57 lines, the median changed-files count is 2, the median commit count is 1, and 71.2% of PRs come from OWNER, MEMBER, or COLLABORATOR author associations.

Agent	Enriched PRs	Median Churn	Median Files	Median Commits	Owner/Member/Collaborator Rate
Claude Code	117	561	7	2	47.9%
Cursor	280	120	3	2	60.0%
OpenAI Codex	1,606	47	2	1	74.9%

Claude Code PRs in the enriched sample are larger by median churn and changed files. OpenAI Codex PRs are smaller and more often come from owner/member/collaborator authors. This reinforces the need to control for PR complexity and author association.

Task Categories

Task categories were assigned with lightweight keyword rules over PR titles and limited body text. They are useful as descriptive strata and controls, not as gold-label annotations.

Task Category	PRs	Merge Rate	Human Review Rate	Approval Rate	Human Discussion Rate
Feature	9,988	75.4%	11.4%	7.9%	17.4%
Bugfix	5,020	70.6%	15.3%	11.4%	25.7%
Docs	3,549	85.0%	12.9%	10.9%	16.5%
Test	3,110	80.4%	6.3%	4.7%	9.9%
CI/build	1,892	77.5%	12.9%	10.4%	17.8%
Dependency	1,411	76.0%	15.5%	12.0%	21.6%
Refactor	927	71.0%	15.7%	12.1%	21.6%
Security	259	64.1%	25.5%	21.2%	37.5%
Performance	205	54.6%	14.6%	10.7%	24.9%

Security, feature, and bugfix PRs receive more review activity than test or documentation PRs. Documentation and test PRs merge at higher rates, while performance and security PRs have lower merge rates.

Controlled Results — Review and Discussion Outcomes

The strongest model uses the GitHub-enriched, human-account-linked, non-extreme sample. It controls for agent, task category, account age, followers, GitHub total churn, changed files, commits, author association, PR month, and repository-clustered standard errors.

Outcome	Effect	OR	p
---------	--------	----	---

Human review	Account age, per SD	0.949	0.679
Human review	GitHub total churn, log1p per SD	0.627	<0.001
Human review	GitHub commits, log1p per SD	1.926	<0.001
Approval	Account age, per SD	0.925	0.582
Approval	GitHub total churn, log1p per SD	0.659	<0.001
Approval	GitHub commits, log1p per SD	1.537	<0.001
Human discussion	Account age, per SD	0.970	0.772
Human discussion	GitHub total churn, log1p per SD	0.780	0.018
Human discussion	GitHub commits, log1p per SD	1.730	<0.001

Account age does not predict human review, approval, or discussion after controls. PR complexity is more important: higher churn lowers review/approval/discussion odds, while more commits are associated with more review activity.

Controlled Results — Merge Success

Among closed PRs, account age was not significant. The strongest predictors were PR complexity and author association.

Effect	OR	p
Account age, per SD	1.132	0.248
GitHub total churn, log1p per SD	0.635	<0.001
GitHub commits, log1p per SD	1.292	0.006
OWNER vs CONTRIBUTOR	2.453	<0.001
NONE vs CONTRIBUTOR	0.046	<0.001
Claude Code vs OpenAI Codex	1.875	0.122
Cursor vs OpenAI Codex	1.072	0.788

Agent effects were not significant in this controlled model. Author association is a strong trust signal: owner-like authors (OWNER vs CONTRIBUTOR OR = 2.453) have much higher merge odds, while authors with no repository association (NONE vs CONTRIBUTOR OR = 0.046) have much lower merge odds.

Controlled Results — Review Duration

Review duration was modeled as log-hours to first human review among PRs that received a human review. In the non-extreme human-linked sample, the median time to first human review was 3.60 hours.

Effect	Duration Ratio	p
Account age, per SD	0.984	0.901
GitHub total churn, log1p per SD	1.339	0.082
GitHub commits, log1p per SD	1.557	0.001
NONE vs CONTRIBUTOR	7.834	<0.001
Claude Code vs OpenAI Codex	0.751	0.473
Cursor vs OpenAI Codex	0.724	0.230

Account age and agent choice do not robustly predict review duration. More commits are associated with longer time to first human review, and authors with no GitHub association to the repository wait substantially longer (Duration Ratio = 7.834).

Interim interpretation: developer experience measured by account age is not a robust independent predictor of agent-assisted PR outcomes. Outcomes are better explained by PR scope, reviewability, author standing, repository context, and high-volume author-repository behavior.

4.5 Dataset Imbalance and Robustness

Status: in progress.

Dataset imbalance is a cross-cutting concern. Codex is heavily overrepresented, and some agents appear in fewer repositories. RQ1 checks equal-agent downsampling and within-repository sentiment. RQ2 controls for repository context and uses cluster-robust standard errors. RQ3 directly compares agents within the same repositories. RQ4 shows that high-volume author-repository pairs can strongly affect naive comparisons.

Area	Main Raw Pattern	Controlled or Normalized Interpretation
Sentiment	Method-sensitive rankings	Partially robust; some rankings change after normalization
Review engagement	Codex low coverage; Copilot high scrutiny	Partially robust after repo/task/language/complexity controls
Within-repo acceptance	Cursor strongest; Copilot weakest	Stronger; stable across $k=5/k=10$ and adjusted models
Merge speed	Cursor fastest; Devin slowest	Stronger; stable in stratified Cox models
Developer experience	Raw differences suggest possible effects	Not robust after controls
Author/repo context	Large raw agent differences	Strong contextual explanation

5. Discussion

Status: partial. The final report should expand this into a stronger synthesis and explicitly connect it to related work.

The current results support the project's central claim: AI coding agent performance is multi-dimensional. Different agents rank differently depending on whether the outcome is sentiment, review coverage, scrutiny, approval, within-repository acceptance, merge speed, task specialization, PR complexity, or author context.

5.1 Convergence Across Sentiment, Engagement, and Outcomes

Across RQ1, RQ2, and RQ3, similar patterns appear at the extremes. GitHub Copilot ranks last under all sentiment methods, draws the most scrutiny and change requests when reviewed, and has the weakest within-repository acceptance profile. Cursor shows the opposite pattern: warm sentiment, low scrutiny with strong raw low-friction approval, and the strongest within-repository acceptance and fastest merge profile.

This convergence matters because it does not come from one metric alone. Sentiment, review behavior, and outcome models independently point toward a trusted-vs-distrusted ordering for some agents.

5.2 Coverage Reframes the Codex Story

OpenAI Codex has strong formal review and outcome signals in some analyses, but RQ2 shows that Codex PRs receive very little human review. Therefore, Codex's strong outcome numbers should not automatically be interpreted as strong human approval. Some of its success may occur through low-review or no-review paths, possibly because Codex is concentrated in smaller changes, trusted author contexts, or high-volume author-repository pairs.

5.3 Context Matters More Than a Simple Leaderboard

The developer-context results add a corrective to raw agent comparisons. Developer account age does not independently explain outcomes after controls. Instead, author association, churn, commit count, and high-volume author-repository behavior explain more of the variation. This suggests that OSS communities evaluate AI-assisted PRs through ordinary maintenance signals: trusted author standing, small/reviewable changes, and repository-specific norms.

The final report should avoid saying one agent is universally best. A more defensible claim is that agent differences exist, but they are filtered through repository context, task type, PR complexity, author trust, and review coverage.

6. Interim Conclusions

At this stage, the evidence suggests that AI coding agents differ in reviewer response and PR outcomes, but the differences depend heavily on measurement method and context.

- Claude Code has the strongest general sentiment profile, but its sample is smaller.
- OpenAI Codex and Cursor perform better under code-review-specific and formal review signals.
- Cursor shows the strongest within-repository acceptance and merge-speed profile.
- GitHub Copilot receives the most scrutiny among reviewed PRs and has the weakest within-repository acceptance profile.
- Devin receives review fastest but merges slower in within-repository merge-speed models.
- Developer account age is not a robust independent predictor after controls.
- Author association, PR scope, complexity, and high-volume author-repository concentration are major contextual factors.

The current interim conclusion is that OSS acceptance of AI-assisted PRs depends less on the agent alone and more on whether the PR is reviewable, appropriately scoped, submitted by a trusted project participant, and situated in a repository context where the author has standing.

7. Threats to Validity

Status: in progress. These will be expanded in the final report.

The project is observational, so results should not be interpreted causally. Sentiment is only a proxy for reviewer reception, and different sentiment tools measure different constructs. Human-reviewed PRs are not a random sample of all PRs, so review-engagement measures are affected by coverage selection. Task labels are approximate. The GitHub-enriched RQ4 subset does not cover the full dataset. Account age, follower count, and author association are imperfect proxies for experience and trust. Agent labels may also represent different levels of human involvement. Finally, Codex overrepresentation, high-volume author-repository pairs, and small Claude Code samples limit some comparisons.

8. References

Status: working bibliography; citation details still need final verification and formatting.

Ahmed, T., Bosu, A., Iqbal, A., & Rahimi, S. (2017). SentiCR: A customized sentiment analysis tool for code review interactions. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*.

Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *OOPSLA*.

Calefato, F., Lanubile, F., Maiorano, F., & Novielli, N. (2017). Sentiment polarity detection for software development. <https://arxiv.org/abs/1709.02984>

Gousios, G., Pinzger, M., & van Deursen, A. (2014). An exploratory study of the pull-based software development model. In *Proceedings of ICSE*.

Hugging Face. (2026). hao-li/AIDev dataset card. <https://huggingface.co/datasets/hao-li/AIDev>

Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of ICWSM*. <https://doi.org/10.1609/icwsml.v8i1.14550>

Imai, S. (2022). Is GitHub Copilot a substitute for human pair-programming? An empirical study. In *Proceedings of ICSE Companion*.

Islam, M. R., & Zibran, M. F. (2018). SentiStrength-SE: Exploiting domain specificity for improved sentiment analysis in software engineering text. *Journal of Systems and Software*, 145, 125-146. <https://doi.org/10.1016/j.jss.2018.08.030>

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2014/2016). The promises and perils of mining GitHub. *MSR / Empirical Software Engineering*.

Li, H., Zhang, H., & Hassan, A. E. (2025). The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering. <https://arxiv.org/abs/2507.15003>

Li, H., Zhang, H., & Hassan, A. E. (2026). AIDev: Studying AI coding agents on GitHub. <https://arxiv.org/abs/2602.09185>

Novielli, N., Calefato, F., Lanubile, F., & Serebrenik, A. (2021). Assessment of off-the-shelf SE-specific sentiment analysis tools: An extended replication study. *Empirical Software Engineering*, 26, Article 77. <https://doi.org/10.1007/s10664-021-09960-w>

Ogenrwot, D., & Businge, J. (2026). How AI coding agents modify code: A large-scale study of GitHub pull requests. <https://arxiv.org/abs/2601.17581>

Rahman, S., Rabbi, M. F., & Zibran, M. (2026). A task-level evaluation of AI agents in open-source projects. <https://arxiv.org/abs/2602.02345>

Rigby, P. C., & Bird, C. (2013). Convergent contemporary software peer review practices. In *Proceedings of ESEC/FSE*.

Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Extended Abstracts*.

9. Updated Timeline and Remaining Work

Final report due: June 28, 2026

Poster presentation: before June 28; exact class date still needs confirmation

9.1 Remaining Work by Member

Member	Done	Remaining
Amrinder	RQ1 sentiment analysis and draft complete	Trim final section, polish figures/captions, verify sentiment references
Gabe	RQ2 framework, models, tables, and descriptive stats complete	Connect scrutiny/engagement with RQ3 merge speed, flush out technical details
Luke	RQ3 within-repo analysis and notebook complete	Integrate with discussion, help with merge-speed synthesis and final tables
Kunwarbir	RQ4 scope and GitHub-enriched initial models complete	Finish robustness analysis, polish tables, write final RQ4 narrative
All	Shared framing and interim synthesis drafted	Final discussion, limitations, references, poster, PDF/Markdown formatting, adjusting scope/length based on professor feedback

9.2 Gantt Chart: June 8 to June 28, 2026

Task	Owner	Jun 8-14	Jun 15-21	Jun 22-28
Finalize RQ4 experience + imbalance analysis	Kunwarbir	Active	Polish	-
Write and integrate RQ4 section	Kunwarbir	Active	Polish	-
Verify and format	Gabriel / all	Active	Active	Polish

related-work references				
Trim RQ1/RQ2/RQ3 sections	Amrinder / Gabe / Luke	Active	Polish	-
Test scrutiny-to-speed link between RQ2 and RQ3	Gabe / Luke	Active	Active	Polish
Polish figures, captions, and tables	All	Active	Active	Polish
Discussion synthesis and limitations	All	Start	Active	Polish
Final conclusion and executive summary	All	-	Active	Polish
Poster / presentation preparation	All	-	Active	Milestone
Final formatting, references, PDF conversion	All	-	Start	Active
Final team review and submission	All	-	-	Milestone

9.3 Final Deliverables Checklist

Deliverable	Status
Interim report Markdown in GitHub repository	Ready after team review
Final report PDF	Remaining; due June 28
Final report Markdown in GitHub repository	Remaining; due June 28
Poster / presentation	Remaining; before June 28